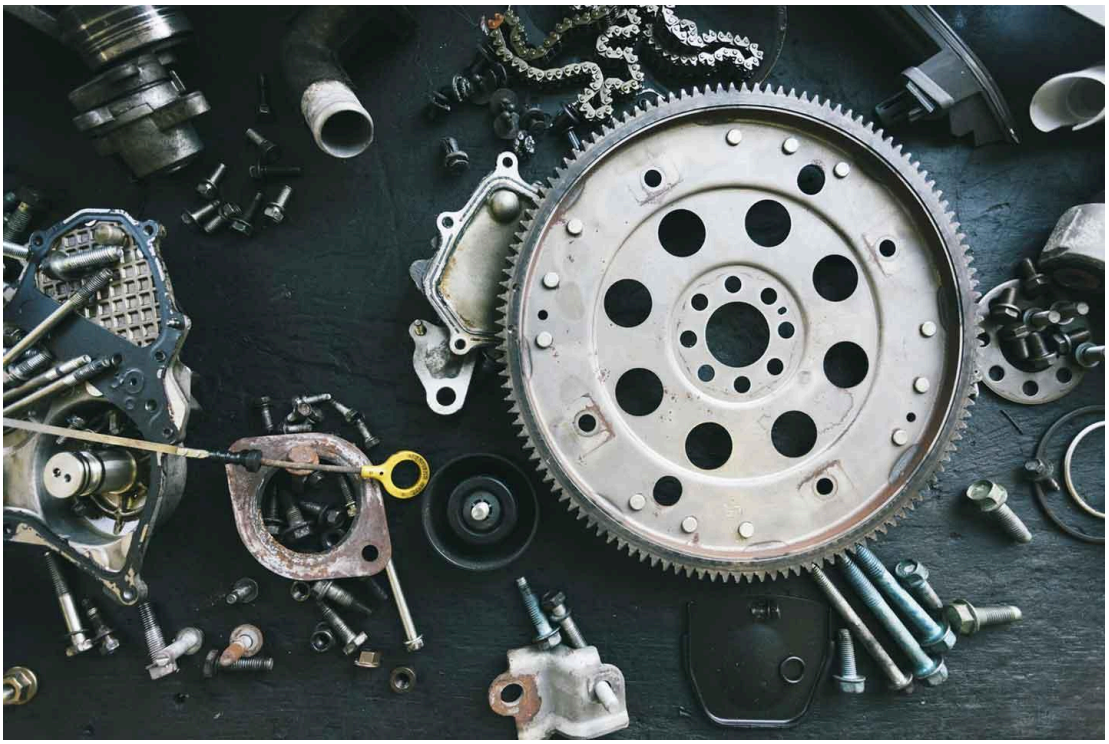
Ionic v8

Segment



Progresa
Una formación de futuro

ÍNDICE

Introducción **3**
Preparación **3**
Componente 3
Icono 3
Json 4
Ejemplo normal **4**
Visualización 4
Explicación 6
Estilos 8
Lógica 8
Ejemplo desplegable **11**
Visualización 11
Estilos 13

Introducción

El componente Segments (<ion-segment>) proporciona un control de nivel superior para alternar entre diferentes vistas o categorías dentro de un mismo contenido. Es similar a las pestañas (Tabs), pero se utiliza comúnmente para la navegación interna y la filtración de contenido dentro de una página.

El componente actúa como un contenedor para múltiples botones de segmento (<ion-segment-button>). Cada botón de segmento representa una vista o categoría que el usuario puede seleccionar.

La documentación oficial de Segments se encuentra en: https://ionicframework.com/docs/api/segment

Preparación

Componente

Para empezar a trabajar con el modal vamos a necesitar una página donde usarlo. Creamos la página en la **terminal**:

```
ionic g page pages/Segments --spec=false
```

Icono

Añadimos en nuestro **app.component.ts** el icono que queremos usar para el menú en la lista de iconos(addIcons).

```
addIcons({grid, … , calendarNumber, batteryHalf});
```

JSON

Añadimos en nuestro **menu.json** el nuevo elemento que queremos que aparezca tanto en el menú como en la página de inicio.

```
,{
 "nombre": "Segments",
 "ruta": "/segments",
 "icono": "battery-half"
}
```

EJEMPLO NORMAL

Usaremos un ejemplo donde se tiene un segment con tres botones: "Amigos", "Álbumes" y "Configuración".

VISUALIZACIÓN (HTML)

Cargamos el contenido del segmento en nuestra página segments.page.html, típicamente dentro de una cabecera o barra de herramientas, y luego mostramos el contenido condicionalmente.

```
<ion-header>
 <ion-toolbar>
   <ion-title>Segment Example</ion-title>  </ion-toolbar>
 <ion-toolbar>
          <ion-segment                    [value]="segmentValue"
(ionChange)="segmentChanged($event)"      color="secondary"    >
     <ion-segment-button value="friends">
       <ion-label>Amigos</ion-label>
     </ion-segment-button>
     <ion-segment-button value="albums">
       <ion-label>Álbumes</ion-label>
     </ion-segment-button>
```

```
      <ion-segment-button value="settings">
        <ion-label>Configuración</ion-label>
      </ion-segment-button>
    </ion-segment>
  </ion-toolbar>
</ion-header>

<ion-content class="ion-padding">
  @if (segmentValue === 'friends'){
    <div>
    <h2>Lista de Amigos</h2>
    <p>Aquí verás tus contactos.</p>
  </div>
  }
  @if (segmentValue === 'albums'){
    <div>
    <h2>Mis Álbumes</h2>
    <p>Fotos y colecciones.</p>
  </div>
  }
  @if (segmentValue === 'settings'){
  <div>
    <h2>Ajustes de la Aplicación</h2>
    <p>Configura tu cuenta.</p>
  </div>
  }
</ion-content>
```

Explicación (HTML)

<ion-segment>: Contenedor principal de los botones de segmento.

[value]="segmentValue": Define qué botón está seleccionado actualmente. Se enlaza a una propiedad TypeScript.

(ionChange)="segmentChanged($event)": Evento que se dispara cuando el usuario selecciona un botón diferente. Permite capturar la nueva selección.

<ion-segment-button value="friends"> : Define cada botón dentro del segmento.

value="friends": Define el valor único del botón, que será el valor de salida en el evento ionChange y el valor usado para el binding en [value].

EJEMPLO (LÓGICA TYPESCRIPT)

En el componente TypeScript, definimos el valor inicial del segmento y la función para manejar el cambio de selección.

```
// Inicializamos la variable con el valor por defecto
public segmentValue: string = 'friends';


// Función que se dispara al cambiar el segmento
segmentChanged(event: any) {
 // El evento contiene el nuevo valor seleccionado
 this.segmentValue = event.detail.value;
 console.log('Segmento seleccionado:', this.segmentValue);
}
```

Explicación (TypeScript)

1. `public segmentValue: string = 'friends';:` Esta variable almacena la vista actual. Inicialmente, se establece en 'friends' para que el primer botón aparezca seleccionado por defecto.
2. `segmentChanged(event: any):` Esta función se ejecuta cada vez que el usuario pulsa un botón.
3. `event.detail.value:` Contiene el valor (value) del botón que el usuario acaba de seleccionar (ej: 'albums'). Este valor se usa para actualizar segmentValue, lo que a su vez activa la renderización condicional del contenido en el HTML (@if).

Segmentos y Query Parameters

En este enfoque, el componente `SegmentsPage` hará tres cosas:

1. Leer el parámetro de consulta (`segment`) de la URL al inicializarse.
2. Actualizar el valor del `ion-segment` con ese parámetro.
3. Cambiar el parámetro de consulta en la URL (sin navegar fuera) cada vez que se presiona un botón del segmento.

Ejemplo estructurado con los siguientes archivos:

1. ***`main.ts`: Habilitar el binding de inputs.***
2. ***`segments.page.ts`: Componente padre que maneja el Segment y actualiza la URL.***
3. ***`segment-content.component.ts`: Componente hijo que recibe el valor vía `@Input` y usa `ngOnChanges`.***

Habilitar withComponentInputBinding (app.config.ts)

Para que el router vincule automáticamente los query parameters (o parámetros de ruta) a los @Input del componente, debes configurar el proveedor de rutas.

```
import { bootstrapApplication } from '@angular/platform-browser';
import {
 RouteReuseStrategy,
 provideRouter,
 withPreloading,
 PreloadAllModules,
 withComponentInputBinding
} from '@angular/router';
import { IonicRouteStrategy, provideIonicAngular } from
'@ionic/angular/standalone';
```

```
import { routes } from './app/app.routes';

import { AppComponent } from './app/app.component';

import {provideHttpClient, withFetch} from "@angular/common/http";


bootstrapApplication(AppComponent, {

 providers: [

   { provide: RouteReuseStrategy, useClass: IonicRouteStrategy },

   provideIonicAngular(),

   provideRouter(routes, withPreloading(PreloadAllModules),
withComponentInputBinding()),

   provideHttpClient(withFetch())

 ],

});
```

Componente Padre: segments.page.ts

El cambio clave aquí es que el constructor y el ngOnInit se simplifican enormemente, ya que el router inyecta el valor de la URL en la propiedad @Input() antes de que se llame a ngOnInit. Usaremos ngOnChanges para reaccionar a la primera carga y a los cambios de la URL.

```
import {Component, inject, Input, OnChanges, OnInit, SimpleChanges} from
'@angular/core';

import {CommonModule} from '@angular/common';

import {FormsModule} from '@angular/forms';

import {IonContent, IonHeader, IonTitle, IonToolbar} from
'@ionic/angular/standalone';

import {Router} from "@angular/router";


interface CategoryView {

 value: string;

 label: string;

}
```

```
@Component({
 selector: 'app-segments',
 templateUrl: './segments.page.html',
 styleUrls: ['./segments.page.scss'],
 standalone: true,
 imports: [IonContent, IonHeader, IonTitle, IonToolbar, CommonModule,
FormsModule]
})
export class SegmentsPage implements OnChanges {
// Implementamos OnChanges para reaccionar al cambio en el @Input

 // El @Input recibe directamente el valor del query parameter 'category'.
 @Input() category!: string;

 public categories: CategoryView[] = [
   { value: 'monsters', label: 'Monstruos' },
   { value: 'materials', label: 'Materiales' },
   { value: 'equipment', label: 'Equipamiento' },
   { value: 'creatures', label: 'Criaturas' },
   { value: 'treasure', label: 'Tesoro' }
 ];

 // currentCategory ahora solo se usa para enlazar el [value] del ion-segment
 public currentCategory: string = this.categories[0].value;
 private validCategories = this.categories.map(v => v.value);
 private readonly router: Router = inject(Router);


 // ngOnChanges reacciona cuando el router actualiza el @Input(category)
 ngOnChanges(changes: SimpleChanges): void {
   if (changes['category'] && changes['category'].currentValue) {
     const newCategory = changes['category'].currentValue;
```

```
      // Sincronización del segmento: El valor del router actualiza la variable de UI
      if (this.validCategories.includes(newCategory)) {
        this.currentCategory = newCategory;
      }

      // Si el valor es inválido o se carga sin él, forzamos el valor por defecto
      if (!newCategory || !this.validCategories.includes(newCategory)) {
        this.updateQueryParam(this.categories[0].value);
      }
    }
  }

  //Se dispara al cambiar el segmento: actualiza la URL y fuerza el cambio de @Input.
  segmentChanged(event: any) {
    const newCategory = event.detail.value;

    if (newCategory) {
      // Escribir el nuevo valor en la URL
      this.updateQueryParam(newCategory);
      // El cambio de URL reejecutará ngOnChanges en este componente y el componente hijo
    }
  }

  private updateQueryParam(category: string) {
    this.router.navigate([], {
      queryParams: { category: category },
      queryParamsHandling: 'merge'
    });
  }
}
```

HTML del Componente Padre (segments.page.html)

El HTML se mantiene casi idéntico. Nota que le pasamos el currentCategory local al hijo.

```
<ion-header>
 <ion-toolbar>
   <ion-title>Hyrule Compendium (Padre con @Input URL)</ion-title>
 </ion-toolbar>

 <ion-toolbar>
   <ion-segment
     [value]="currentCategory"
     (ionChange)="segmentChanged($event)"
     color="secondary"
   >
     @for (cat of categories; track cat.value) {
       <ion-segment-button [value]="cat.value">
         <ion-label>{{ cat.label }}</ion-label>
       </ion-segment-button>
     }
   </ion-segment>
 </ion-toolbar>
</ion-header>

<ion-content class="ion-padding">
 <app-segment-content
[category]="currentCategory"></app-segment-content>
</ion-content>
```

COMPONENTE HIJO: SEGMENT-CONTENT.COMPONENT.TS

Este componente se enfoca en la lógica de la llamada a la API y el renderizado, reaccionando únicamente al `@Input` de la categoría.

```
import {Component, inject, Input, OnChanges, SimpleChanges}
from '@angular/core';

import {IonItem, IonLabel, IonList, IonNote, IonSpinner,
IonText} from "@ionic/angular/standalone";

import { TitleCasePipe, UpperCasePipe} from "@angular/common";

import {DataService} from "../../services/data.service";

export interface CompendiumItem {

 id: number;

 name: string;

 description: string;

 image: string;

}

export interface CompendiumResponse {

 data: CompendiumItem[];

}


@Component({

 selector: 'app-segment-content',

 templateUrl: './segment-content.component.html',

 styleUrls: ['./segment-content.component.scss'],

 imports: [

   IonSpinner,

   UpperCasePipe,
```

```
    IonList,
    IonItem,
    IonLabel,
    TitleCasePipe,
    IonNote,
    IonText
  ],
  standalone: true
})
export class SegmentContentComponent  implements OnChanges {


  // @Input para recibir la categoría desde el componente padre
  // El padre le pasa el valor que lee del query parameter
  @Input() category!: string;
  private readonly dataService: DataService =
inject(DataService);

  public items: CompendiumItem[] = [];
  public isLoading: boolean = false;
  public categoryInput: string = ''; // Copia local para
mostrar en el template


  // ngOnChanges se dispara cuando cualquier @Input cambia
  ngOnChanges(changes: SimpleChanges): void {
```

```
    // Verificamos si el input 'category' ha cambiado
    if (changes['category'] &&
changes['category'].currentValue) {
      const newCategory = changes['category'].currentValue;
      this.categoryInput = newCategory; // Actualizamos la
copia local

      // Llamamos a la función de API solo si el valor es nuevo
      if (!changes['category'].firstChange) {
        this.fetchCategoryData(newCategory);
      }
      // En la primera carga, también llamamos a la API
      if (changes['category'].firstChange && newCategory) {
        this.fetchCategoryData(newCategory);
      }
    }
  }

  // Realiza la llamada a la API para la categoría
seleccionada.

  private fetchCategoryData(category: string): void {
    this.isLoading = true;
    this.items = [];
```

```
    this.dataService.getCategories(category)
      .subscribe({
        next: (response) => {
          this.items = response.data;
          this.isLoading = false;
        },
        error: (err) => {
          console.error('Error fetching data:', err);
          this.isLoading = false;
        }
      });
  }
}
```

Para esto vamos a necesitar una función en el servicio que traiga los elementos:

```
private API_URL = 'https://botw-compendium.herokuapp.com/api/v3/compendium/category/';
getCategories(category: string): Observable<CompendiumResponse> {
 return this.http.get<CompendiumResponse>(this.API_URL + category);
}
```

COMPONENTE HIJO: SEGMENT-CONTENT.COMPONENT.HTML

```
<div class="ion-text-center ion-padding">
 @if (isLoading) {
   <ion-spinner name="crescent" color="primary"></ion-spinner>
   <p>Cargando {{ categoryInput | uppercase }}...</p>
 }
</div>
@if (!isLoading && items.length > 0) {
 <ion-list>
   <ion-item lines="full" color="light">
        <ion-label>Lista de Elementos ({{ categoryInput | uppercase
}})</ion-label>
   </ion-item>
   @for (item of items; track item.id) {
     <ion-item [button]="true" [detail]="true">
          <img [src]="item.image" alt="Imagen de {{ item.name }}"
class="item-thumbnail">
       <ion-label>
         <h2>{{ item.name | titlecase }}</h2>
         <p class="text-overflow">{{ item.description}}</p>
       </ion-label>
       <ion-note slot="end">{{ item.id }}</ion-note>
     </ion-item>
   }
 </ion-list>
} @else if (!isLoading) {
 <ion-text color="medium">
       <p class="ion-text-center ion-padding">No se encontraron
elementos o la categoría no está disponible.</p>
 </ion-text>
}
```

COMPONENTE HIJO: SEGMENT-CONTENT.COMPONENT.SCSS

```scss
.item-thumbnail {
 width: 50px;
 height: 50px;
 margin-right: 10px;
 object-fit: contain;
}
.text-overflow{
 overflow: hidden;
 text-overflow: ellipsis;
 white-space: nowrap;
}
```

Resumen del Flujo de Datos

1. El usuario hace clic en el segmento (ej: Materiales).
2. El método **segmentChanged()** en **SegmentsPage** se ejecuta y llama a **updateQueryParam('materials')**.
3. **updateQueryParam** usa el Router para cambiar la URL a .../ruta?category=materials.
4. El router (gracias a withComponentInputBinding) detecta el cambio en el query parameter category y actualiza el @Input('category') del componente hijo (**SegmentContentComponent**).
5. El lifecycle hook **ngOnChanges** en **SegmentContentComponent** se activa.
6. **ngOnChanges** llama a **fetchCategoryData('materials')**, que realiza la llamada HTTP a la API.
7. El hijo muestra la lista de elementos actualizada.